

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Report

on laboratory work no. 1

Network Programming

Topic: Concurrency in Practice

Done by:
st. gr. FAF-242, **Vasile Brînză**
st. gr. FAF-242, **Gabriela Bîtca**

Checked by:
Cristofor Fiștic, *university assistant*
DISA, FCIM, UTM

Chișinău 2026

TABLE OF CONTENTS

PURPOSE OF THE WORK *4*

1 HARDWARE AND TEST SETUP *5*

1.1 The test machine **5**

1.2 Program and data **5**

2 TASK 1: ONE FILE *7*

2.1 Task 1a - counting with threads **7**

2.2 Task 1a - results **7**

2.3 Task 1b - three ways to count **8**

2.4 Task 1b - results **8**

2.5 Task 1c - producer and consumers **9**

2.6 Task 1c - results **10**

2.7 Answers to the questions of Task 1 ..... **11**

3 TASK 2: MANY FILES ON SLOW STORAGE *14*

3.1 Setup and implementation **14**

3.2 Results **14**

3.3 Answers to the questions of Task 2 ..... **15**

4 ANSWERS TO THE CONTROL QUESTIONS *16*

4.1 Physical core and logical processor **16**

4.2 No line counted twice or skipped **16**

4.3 How unsynchronised counters lose updates **16**

4.4 Why per-thread counters beat a lock **16**

4.5 I/O-bound: USB drive and network server **16**

4.6 Equal parts in advance or a queue **17**

CONCLUSIONS *18*

REFERENCES *19*

PURPOSE OF THE WORK

The purpose of this laboratory work [5] is to measure how the number of threads, and the way the work is divided among them, change the running time and the memory of a real workload. By the end of the work the goals are:

- to count the negative, positive and zero numbers in a file of 50,000,000 integers with 1 to 64 threads and to find where the speed-up stops (Task 1a);
- to observe a race condition and to measure what it costs to fix it with a lock and with per-thread counters (Task 1b);
- to compare dividing the file in advance with a producer–consumer queue (Task 1c);
- to decide from measurements whether the work is CPU-bound or I/O-bound when the files are on a slow USB drive (Task 2).

The program is one C# (.NET 8) console application; every task and configuration is selected by command-line parameters. All measurements in this report were made on the same laptop, described in Chapter 1. The source code, with instructions to build and run every task, is in the Git repository [1]: `https://github.com/kynexi/pr-labs` (folder `lab1`). The raw measurements are CSV files in `lab1/results` (for Tasks 1a and 1c, the versions from commit `bfdbf27`).

1 HARDWARE AND TEST SETUP

1.1 The test machine

The values below are the ones reported by Windows (PowerShell, `Get-CimInstance`) on the machine used for every measurement. The Ryzen 7 8845HS has 8 physical cores with two hardware threads each (SMT), so Windows sees 16 logical processors.

Table 1.1 - Hardware and software of the test machine

Item	Value
Computer	Lenovo 83D3 laptop
CPU model	AMD Ryzen 7 8845HS with Radeon 780M Graphics
Physical cores / logical processors	8 / 16
RAM	32 GB installed (27.8 GiB available to Windows)
Internal disk	WD PC SN740 SDDPMQD-1T00-1101, 1 TB NVMe SSD
USB drive	Kingston DataTraveler 3.0, 64 GB
USB version	USB 3.0 drive (from the model name), connected to an AMD USB 3.10 host controller; the negotiated link speed was not confirmed
Operating system	Windows 11 Pro, build 26200
Language and runtime	C# with .NET SDK 8.0.412

1.2 Program and data

The program is in `lab1/csharp`. The first word of the command line chooses what to do: `generate`, `split`, `verify` or `run`. All measurements use a Release build (from that folder):

```
dotnet run -c Release -- generate | split | verify     # data and reference totals
dotnet run -c Release -- run --task 1a --threads 8
dotnet run -c Release -- run --task 1b --variant unsync   # also: lock, local
dotnet run -c Release -- run --task 1c --threads 4
dotnet run -c Release -- run --task 2 --threads 2 --dir K:\
```

`numbers.txt` contains 50,000,000 random integers from −1,000,000 to 1,000,000, one per line (fixed seed 42); its size is about 370 MB. For Task 2 it is split, in order, into 8 files of 6,250,000 numbers (about 44 MB each).

Correctness check. `verify` gives the reference totals: **25,005,087 negative, 24,994,889 positive and 24 zero**, together 50,000,000. Every correct run returns exactly these numbers; only the unsynchronised runs of Task 1b fail the check, which is the point of that task.

How the measurements were taken. The time is measured with a `Stopwatch` around the task only (in Task 1b the loading of the array is not included). The peak memory is the peak working set of the process (`Process.PeakWorkingSet64`) in MB, where 1 MB = 1,048,576 bytes [4]. Tasks 1a, 1c and 2 were run once per thread count; in Task 1b the unsynchronised version was run three times, the others once. Differences of a few percent are inside the single-run noise and are not interpreted.

2 TASK 1: ONE FILE

2.1 Task 1a - counting with threads

The file is divided by bytes into as many parts as there are threads: thread *i* of *n* gets the range from `length * i / n` to `length * (i + 1) / n`. Each thread opens the file itself, reads only its own range in 64 KB blocks, parses the numbers by hand from the bytes and counts them in its own variables. At the end the main thread joins all threads and adds their three counters. Nothing reads the whole file on one thread.

Lines at the border. A border between two ranges usually falls in the middle of a line. The rule is: *a line belongs to the thread whose range contains the first character of the line.* So a thread starts reading one byte before its range and skips everything up to the first newline (that partial line is finished by the previous thread); and it keeps reading until the next line would start at or after the end of its range, so it always finishes its last line, even past the border. Starting one byte early matters: if that byte is a newline, the first line of the thread starts exactly at the border and must not be skipped. In this way no line is counted twice and none is skipped.

```
fs.Position = start > 0 ? start - 1 : 0;   // one byte early
bool skipping = start > 0;                 // skip the partial first line
...
if (skipping) {                      // still inside the partial line
    if (b == '\n') { skipping = false; atLineStart = true; }
    continue; }
if (atLineStart && pos + i >= end) return counts;   // this line is not ours
```

2.2 Task 1a - results

Table 2.1 and Figure 2.1 show the results for 1, 2, 4, 8, 16, 32 and 64 threads. The speed-up is the time with one thread divided by the time with *n* threads. All runs returned the reference totals.

Table 2.1 - Task 1a: counting with threads

Threads	Time (ms)	Speed-up	Peak memory (MB)	Total counted
1	533.7	1.00×	25.9	50,000,000
2	320.9	1.66×	25.8	50,000,000
4	182.0	2.93×	25.8	50,000,000
8	128.0	4.17×	26.6	50,000,000
16	115.9	4.60×	26.7	50,000,000
32	117.6	4.54×	28.1	50,000,000
64	118.7	4.50×	30.2	50,000,000

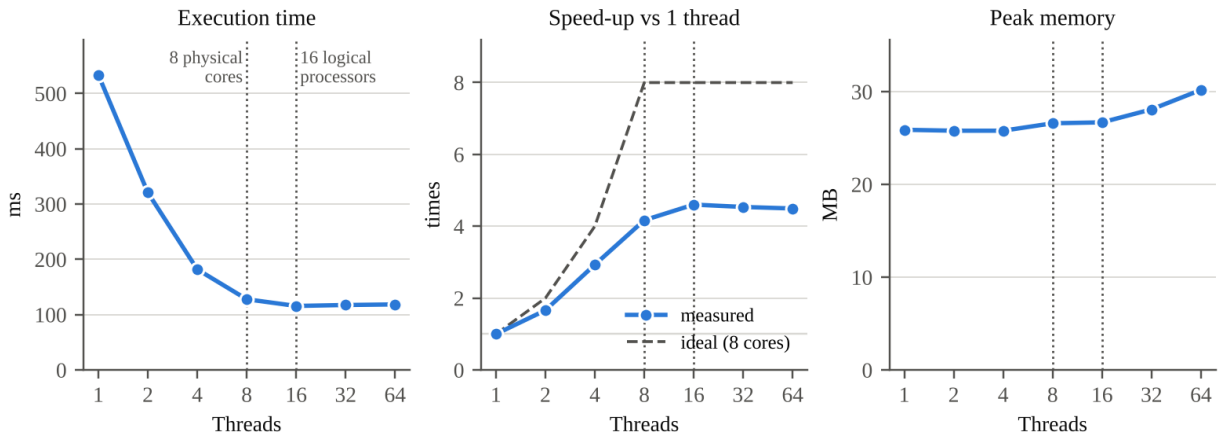


Figure 2.1 - Task 1a: execution time, speed-up and peak memory against the number of threads

2.3 Task 1b - three ways to count

All 50,000,000 numbers are first loaded into an `int[]` array (about 200 MB); this is not measured. Then 16 threads (the number of logical processors) count the signs, each in its own, nearly equal, range of the array. The only difference between the three versions is where the counters are:

- **unsynchronised:** all threads add to the same three shared variables, with no protection;
- **lock:** the same shared variables, but every increment is inside `lock (gate) { ... }` [3];
- **per-thread counters:** every thread counts in its own local variables and stores the result in its own cell of an array; the main thread adds the 16 results after all threads have finished.

```
// 1. unsynchronised: shared variables, no protection
if (number < 0) negative++;
else if (number > 0) positive++;
else zero++;
// 2. lock: the same three lines inside  lock (gate) { ... }
// 3. per-thread: private variables, added together after Join()
```

2.4 Task 1b - results

Table 2.2 lists every run with its totals. Figure 2.2 compares the time and the peak memory of the three versions (the unsynchronised bar is the average of its three runs). All versions use the same 16 threads, so a speed-up relative to one thread is not defined for this task.

Table 2.2 - Task 1b: three ways to count, 16 threads

Version	Run	Time (ms)	Negative	Positive	Zero	Total	Correct
Shared, unsynchronised	**1**	**108.2**	**4,014,923**	**3,578,714**	**24**	**7,593,661**	**No**
Shared, unsynchronised	**2**	**105.9**	**3,531,733**	**3,139,982**	**24**	**6,671,739**	**No**
Shared, unsynchronised	**3**	**104.5**	**3,586,904**	**3,183,801**	**24**	**6,770,729**	**No**
Shared, lock	1	3,095.2	25,005,087	24,994,889	24	50,000,000	Yes
Per-thread counters	1	29.3	25,005,087	24,994,889	24	50,000,000	Yes

The reference totals are 25,005,087 negative, 24,994,889 positive and 24 zero (50,000,000). Peak memory was 230.3–230.9 MB in all five runs.

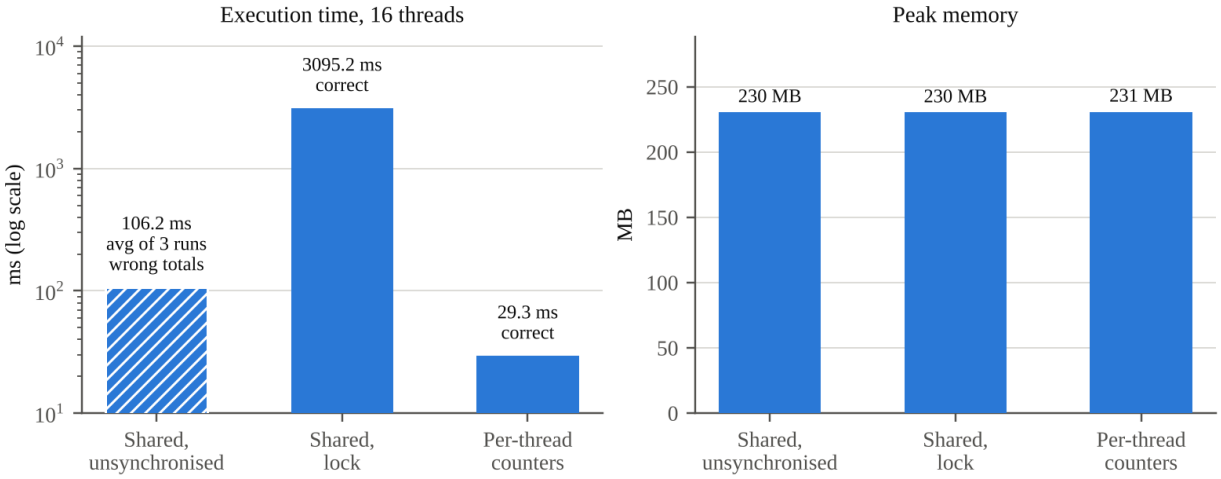


Figure 2.2 - Task 1b: execution time (log scale) and peak memory of the three versions

2.5 Task 1c - producer and consumers

One reader thread reads `numbers.txt` line by line with a `StreamReader` and puts the lines, in batches of 10,000, into a `BlockingCollection<string[]>` that holds at most 100 batches [2]. When the queue is full, `Add` makes the reader wait; when it is empty, the workers wait inside `GetConsumingEnumerable`. At the end of the file the reader calls `CompleteAdding()` (closes the queue); the enumeration of a worker ends when the queue is closed and empty. Each worker parses the lines of its batch with `int.Parse` and counts them in local variables; the main thread adds the results.

```
using var queue = new BlockingCollection<string[]>(100);   // at most 100 batches
// reader:    queue.Add(batch.ToArray());  ...  queue.CompleteAdding();
// workers:   foreach (var b in queue.GetConsumingEnumerable())
//                foreach (var s in b) { int n = int.Parse(s); /* count it */ }
```

2.6 Task 1c - results

Table 2.3 and Figure 2.3 show the results with 1 to 64 workers (the reader is one extra thread). Figure 2.4 puts Task 1c next to Task 1a. All runs returned the reference totals.

Table 2.3 - Task 1c: producer and consumers

Workers	Time (ms)	Speed-up	Peak memory (MB)	Total counted
1	1,552.9	1.00×	194.6	50,000,000
2	1,384.6	1.12×	68.4	50,000,000
4	1,494.4	1.04×	76.9	50,000,000
8	1,992.2	0.78×	118.3	50,000,000
16	1,663.0	0.93×	164.9	50,000,000
32	1,782.8	0.87×	173.6	50,000,000
64	1,988.5	0.78×	189.6	50,000,000

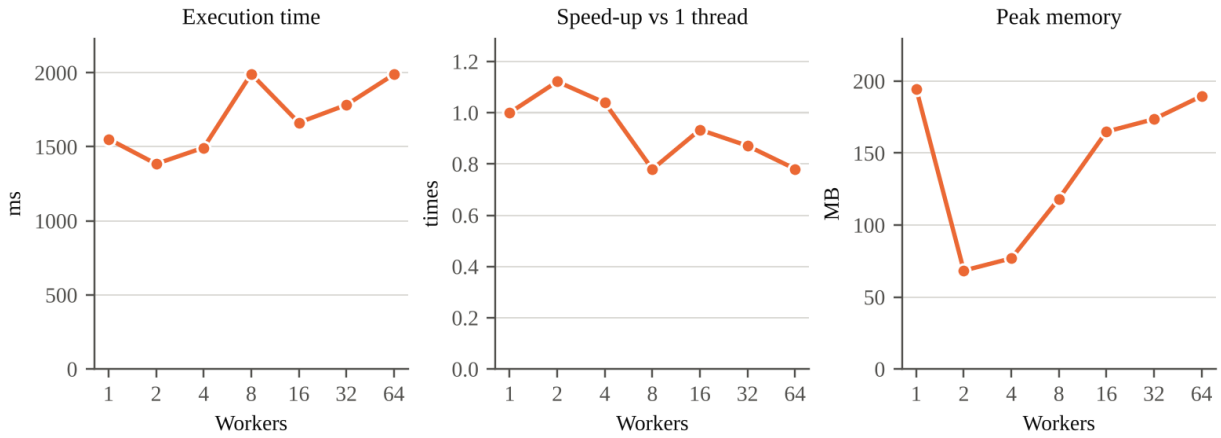


Figure 2.3 - Task 1c: execution time, speed-up and peak memory against the number of workers

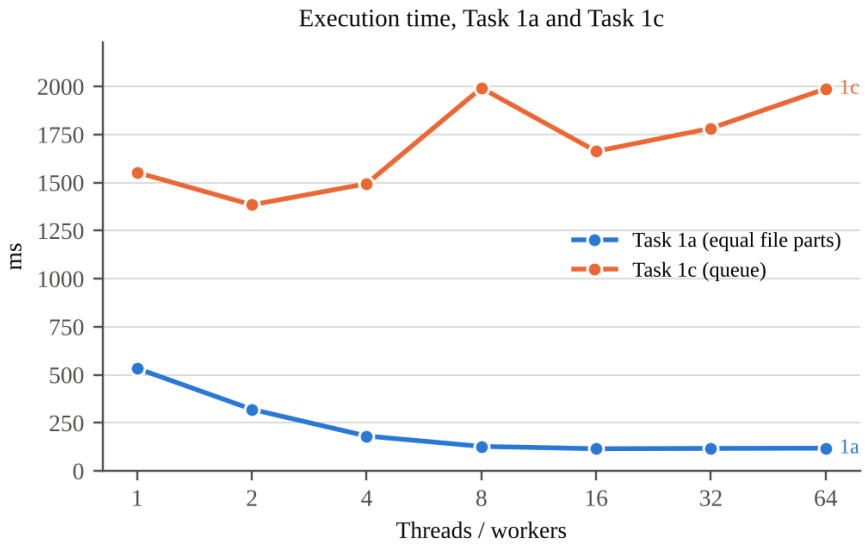


Figure 2.4 - Execution time of Task 1a and Task 1c

2.7 Answers to the questions of Task 1

1. At which number of threads does the speed-up stop growing? Relate it to your physical cores and logical processors.

The speed-up grows up to 8 threads, which is the number of physical cores: the time falls from 533.7 ms to 128.0 ms (4.17×). From 8 to 16 threads, the number of logical processors, it grows only a little more (115.9 ms, 4.60×, 9% less time), and after 16 it is flat. Two threads on the same core share its execution units, so the 8 extra logical processors add much less than 8 more cores would. The speed-up is also far below the ideal 8×: at 8 threads it reaches only 52% of it. Likely reasons are that the threads share the memory and the cache, that a laptop CPU lowers its clock when many cores are busy, and that the whole run is only about 100 ms, so starting and joining the threads is a noticeable part of it. I did not measure which of these is the biggest.

2. Why do 32 and 64 threads not help? Are they slower, and why?

There are only 16 logical processors, so 32 or 64 threads cannot run at the same time; they take turns on the same processors and the work done is the same. They are a little slower: 117.6 ms and 118.7 ms against 115.9 ms at 16 threads (1.5% and 2.4% more). The extra time goes to creating the threads, to switching between them and to opening the file and allocating a read buffer in every thread. The difference is small, close to the noise of a single run, so the safe conclusion is “no gain, no loss to speak of”.

3. Does the peak memory depend on the number of threads? Why?

In Task 1a only very little: from 25.9 MB to 30.2 MB for 1 to 64 threads. The file is never loaded into memory; every thread has only a 64 KB read buffer and its own stack, so one more thread costs about 70 KB. Most of the 26 MB is the .NET runtime itself. In Task 1b the memory is about 230 MB in all versions, because the array of 50,000,000 integers (200 MB) dominates and the threads add nothing

noticeable. In Task 1c the memory depends on the number of workers (68–195 MB) but not in a clean way, see question 5.

4. Rank the three versions of Task 1b by time. Which of them are correct? Explain each result.

By time, from the fastest: **per-thread counters** (29.3 ms), **unsynchronised** (106.2 ms on average) and **lock** (3,095.2 ms). Only the per-thread and the lock versions are correct.

Unsynchronised is wrong. `negative++` is really three steps (read, add one, write back), and the 16 threads do them on the same variable at the same time, so one thread overwrites the increment of another. The three runs counted only 7,593,661, 6,671,739 and 6,770,729 numbers (15%, 13% and 14% of 50,000,000), and the totals are different every time. It is also not the fastest: it is about 3.6 times slower than per-thread counters, most likely because all cores keep writing the same cache line, which has to move between the cores all the time.

Lock is correct, because only one thread at a time can change the counters, but it is the slowest: 106 times slower than per-thread counters and 29 times slower than the unsynchronised version. Every one of the 50,000,000 numbers has to take the same lock, so the threads wait for each other and the counting is in effect serial, with the cost of locking added: about 62 ns of elapsed time per number against 0.6 ns for private counters (all 16 threads together).

Per-thread counters are correct and the fastest: during the counting the threads share nothing, so no waiting and no cache-line traffic; the only shared step is the addition of 16 results at the end.

5. Compare Task 1c with Task 1a. Which one is faster, and why? At which number of workers does Task 1c stop getting faster?

Task 1a is much faster. Its best time is 115.9 ms (16 threads), the best time of Task 1c is 1,384.6 ms (2 workers), 11.9 times more; even Task 1a with one thread (533.7 ms) beats Task 1c with one worker (1,552.9 ms) by 2.9 times. In Task 1a the reading and the parsing are both divided among the threads. In Task 1c a single reader has to read every line, make a string object for it (50,000,000 of them, which the garbage collector must clean up) and put it in the queue, and the workers must parse these strings; the queue itself also costs time for locking and waking threads.

Task 1c stops getting faster at **2 workers**, and the gain is small (1.12×). With one worker the worker is probably the slower side: the queue fills up (the memory is highest, 195 MB) and the reader waits. With two workers the parsing is no longer the limit and the single reader becomes the bottleneck (memory falls to 68 MB, the queue stays short), so more workers only wait for batches. From 8 workers on the time is worse than with one worker (0.78× at 64 workers), because more threads compete for the same queue. Each configuration was run once, so the order of 1, 2 and 4 workers (1,552.9, 1,384.6 and 1,494.4 ms) is inside the noise; only the plateau and the slow-down for many workers are clear.

The memory of Task 1c (68–195 MB) is much higher than in Task 1a, because the queue holds up to 100 batches of 10,000 strings (roughly 50 MB by my estimate) and every line is a new string object that stays in memory until the garbage collector runs. This is also why the values jump from one configuration to the next: they depend on when the collector happens to run.

6. What would happen with batches of a single line? And with a queue that has no size limit?

With batches of a single line every one of the 50,000,000 numbers would go through the queue separately: one lock and possibly one wake-up of a waiting thread per number. In Task 1b a lock alone cost about 62 ns of elapsed time per number, and a queue operation includes such locking and more, while the counting itself with private counters took only about 0.6 ns per number; so the queue overhead would be much bigger than the work and the program would be far slower than with 10,000 lines per batch. With a queue without a size limit nothing stops the reader: it adds lines faster than the workers take them, so the memory grows, and in the worst case the whole file is held in memory as string objects (my estimate: about 2.4 GB for a 370 MB file). The limit of 100 batches makes the reader wait when the workers are slow (back-pressure) and keeps the memory bounded.

3 TASK 2: MANY FILES ON SLOW STORAGE

3.1 Setup and implementation

The 8 split files were copied to the Kingston DataTraveler 3.0 USB drive (drive `K:`) and in this task they are read only from the drive. Before every run the drive was ejected, unplugged and plugged back in, so that no data comes from the Windows file cache. (The Spotlight rule of the task is for macOS; this machine runs Windows.)

A file is the unit of work. A thread reads a whole file in one operation, `File.ReadAllBytes`, into one byte buffer, parses the numbers directly from the bytes (`Utf8Parser`) and counts them. The 8 files are divided among the threads in consecutive groups of as equal size as possible: with 1 thread one thread has all 8 files, with 2 threads 4 + 4, with 4 threads 2 + 2 + 2 + 2.

```
int start = worker * 8 / threads;          // first file of this thread
int end   = (worker + 1) * 8 / threads;    // after the last file
for (int part = start; part < end; part++)
    total += CountFile(PartFile(dir, part + 1));   // read, parse, count
```

3.2 Results

Table 3.1 and Figure 3.1 show one run for each of 1, 2 and 4 threads. All runs returned the reference totals.

Table 3.1 - Task 2: eight files on the USB drive

Threads	Time (ms)	Speed-up	Peak memory (MB)	Total counted
1	4,144.3	1.00×	69.5	50,000,000
2	3,665.8	1.13×	113.6	50,000,000
4	3,641.7	1.14×	246.7	50,000,000

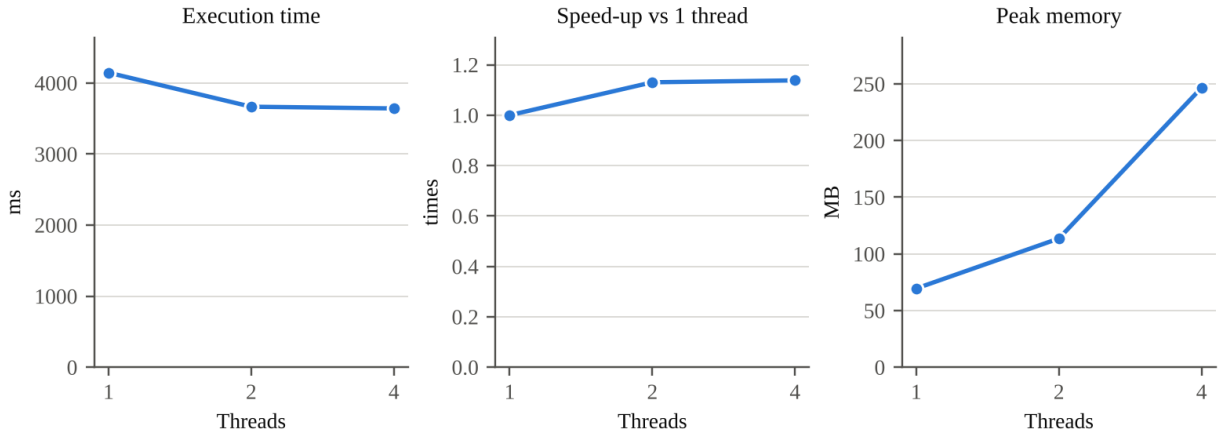


Figure 3.1 - Task 2: execution time, speed-up and peak memory against the number of threads

3.3 Answers to the questions of Task 2

1. Compare the time on one thread with Task 1a on one thread. On the USB drive, is the work CPU-bound or I/O-bound? Support the answer with numbers.

On one thread Task 2 needs 4,144.3 ms and Task 1a, which parses the same 50,000,000 numbers from the internal disk, needs 533.7 ms: the USB version is **7.8 times slower**. The parsing and counting is the same work in both (Task 2 uses a different, equally simple parser), so about 3,610.6 ms of the 4,144.3 ms is time spent waiting for the drive, not computing. That is 87% of the run. For the whole file (about 370 MB) this means a reading speed of only about 89 MB/s, or 102 MB/s if the 533.7 ms of parsing is taken out. This is more than USB 2.0 can deliver (60 MB/s at most), so the drive was working over USB 3, but it is far below the 5 Gbit/s (about 500 MB/s) of a USB 3.0 link, so the flash drive itself is the limit. The work on the USB drive is therefore **I/O-bound**.

Two remarks on this comparison. Task 1a was not run with a cold cache and the file is small for the 27.8 GiB of RAM, so it was most probably read from the Windows file cache and the 533.7 ms is practically pure CPU time, which is what is needed here. And the negotiated speed of the USB link was not read from the system, so I give only the measured reading speed and what follows from it.

2. On the USB drive, do more threads help? Why not, when lecture 1 showed that I/O-bound work benefits from concurrency?

Only a little. Two threads save 478.5 ms (11.5%) and four threads 502.6 ms (12.1%) against one thread; going from 2 to 4 threads gives practically nothing (3,665.8 ms and 3,641.7 ms). The saving of about 478.5 ms is close to the 533.7 ms of parsing: while one thread parses its file, another thread can already wait for the drive, so the parsing is hidden behind the reading. But the reading itself, about 3,610.6 ms, does not become shorter.

Concurrency helps I/O-bound work when the waits are independent and can overlap, for example a server that waits for many different clients or remote machines: while one request waits, others are served. Here every thread waits for the same single device, which delivers a fixed number of bytes per second; more threads only share this speed between them. (Several files read at the same time can also make a flash drive less efficient than one long sequential read, but this was not visible in the measurements.)

The peak memory grows with the number of threads, 69.5, 113.6 and 246.7 MB, because every thread keeps a whole file (about 44 MB) in one buffer: the runtime (about 26 MB) plus 1, 2 or 4 buffers gives about 70, 114 and 202 MB. With 4 threads the measured value is about one buffer more, probably because a buffer of the previous file was not yet freed by the garbage collector.

4 ANSWERS TO THE CONTROL QUESTIONS

4.1 Physical core and logical processor

A physical core is a real processing unit, with its own execution units. A logical processor is a hardware thread that the operating system can schedule. With SMT, one physical core shows two logical processors that share the execution units and the caches of the core, so two threads on one core typically run only 10–30% faster than one, not twice as fast. My CPU has 8 physical cores and 16 logical processors. This shows in Figure 2.1: the speed-up grows up to 8 threads (4.17×), grows only by 10% more at 16 threads (4.60×) and is flat after that.

4.2 No line counted twice or skipped

By the rule "a line belongs to the thread whose range contains its first character" (Section 2.1): every thread skips the partial line at the start of its range and finishes the last line of its range, even after the border. Starting one byte before the range handles the case when the border is exactly at the start of a line. The check is the total: in all 7 runs of Task 1a it was exactly 50,000,000, with the same negative, positive and zero counts as the reference.

4.3 How unsynchronised counters lose updates

`negative++` is not one step but three: (1) the thread reads the value of the counter, (2) adds one, (3) writes the result back. Suppose the counter is 100. Thread A reads 100; thread B also reads 100, before A has written; A writes 101; B writes 101. Two numbers were counted but the counter grew by one: one update is lost. With 16 threads working on the same three variables, this happens all the time, and in my runs only 13–15% of the numbers were counted (Table 2.2).

4.4 Why per-thread counters beat a lock

With a lock, every number must take the same lock, and only one thread can be inside at a time; the others wait, so the work is in effect done one thread at a time, with the cost of taking and releasing the lock for each of the 50,000,000 numbers (about 62 ns of elapsed time per number here). A private counter is a local variable that stays in a register or in the cache of the core (about 0.6 ns per number): no waiting, no traffic between cores. The results are added once at the end, 16 additions instead of 50,000,000 lock operations. Measured: 29.3 ms against 3,095.2 ms.

4.5 I/O-bound: USB drive and network server

I/O-bound means that the program spends most of its time waiting for input or output and not computing, so a faster CPU would not make it faster. For the USB drive the wait is for the flash drive to deliver the bytes (about 87% of the run in Task 2). For a server waiting on the network, the wait is for the

other side of the connection to send or accept data. Concurrency helps in the network case because every request waits for a different remote machine, so many waits can happen at the same time and the server can use the waiting time to serve other requests. It does not help in the USB case because all threads wait for the same device with a fixed speed: the waits cannot be overlapped, only the parsing can (Section 3.3).

4.6 Equal parts in advance or a queue

Dividing in advance (Task 1a, Task 2) needs no synchronisation and no extra thread, so it is simple and very fast, but the parts must cost about the same: if one part is slower, the other threads finish early and wait. It is the better choice when the work can be split evenly and cheaply, as the byte ranges of one file. Taking work from a queue (Task 1c) balances the load automatically, because a free worker takes the next batch, and it lets a producer feed the workers when the data arrives as a stream or must be read in one place; but every batch costs synchronisation, and a single reader can become the bottleneck, as in my results (11.9 times slower than Task 1a). A queue is better when the cost of the pieces is uneven or not known in advance.

CONCLUSIONS

The laboratory work measured the same counting task in four tasks on one machine (Ryzen 7 8845HS, 8 cores, 16 logical processors, NVMe SSD and a USB 3.0 flash drive). The conclusions about what is CPU-bound and what is I/O-bound, and how I know it, are:

- **Task 1a is CPU-bound.** The data comes from fast memory and disk, and the time falls with the number of threads: 533.7 ms with one thread and 128.0 ms with 8 (4.17×), then it stays flat, because there are only 8 physical cores (16 logical processors). More threads than logical processors only add a little overhead.
- **Task 1b is CPU- and synchronisation-bound.** Everything is in memory, so the time shows only the cost of the counters: 29.3 ms with per-thread counters, 3,095.2 ms with a lock, and a wrong result without synchronisation (14% of the numbers counted).
- **Task 1c is limited by one thread, the reader.** Reading lines and making strings is done on one thread, so the best time is 1,384.6 ms with 2 workers, 11.9 times slower than Task 1a, and more workers do not help (0.78× at 64).
- **Task 2 is I/O-bound.** The same numbers need 4,144.3 ms from the USB drive and 533.7 ms from the internal disk (7.8 times more), about 87% of the time is waiting for the drive, and 2 or 4 threads give only 11.5–12.1% less time, because they can hide the parsing but not the reading.

The main lesson is that more threads help only when the limit is the CPU, only up to the number of cores, and only if the threads do not fight over shared data: the lock made the program 106 times slower than separate counters, and a single shared device, the USB drive, cannot be sped up by threads at all.

Limits of these measurements: Tasks 1a, 1c and 2 were run once per configuration (as the task allows), so small differences are not reliable; Task 1a was most probably read from the file cache; the USB link speed was not confirmed; and everything was measured on a single machine.

REFERENCES

1. GitHub repository, folder lab1 (source code, results and instructions). [Online]. Available: https://github.com/kynexi/pr-labs
2. Microsoft, "BlockingCollection<T> Class," .NET API documentation. [Online]. Available: https://learn.microsoft.com/en-us/dotnet/api/system.collections.concurrent.blockingcollection-1
3. Microsoft, "The C# lock statement - synchronize access to shared resources," C# language reference. [Online]. Available: https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/lock
4. Microsoft, "Process.PeakWorkingSet64 Property," .NET API documentation. [Online]. Available: https://learn.microsoft.com/en-us/dotnet/api/system.diagnostics.process.peakworkingset64
5. Laboratory Work 1: Concurrency in Practice, laboratory task, Technical University of Moldova, 2026.